

C/C++

```
/*
Número de equipo: 107, Miguel Amiel, Chen Xu
100525454@alumnos.uc3m.es, 100522395@alumnos.uc3m.es
*/

%{                                     // SECCION 1 Declaraciones de C-Yacc

#include <stdio.h>
#include <ctype.h>                     // declaraciones para tolower
#include <string.h>                   // declaraciones para cadenas
#include <stdlib.h>                   // declaraciones para exit ()

#define FF fflush(stdout);           // para forzar la impresion inmediata
#define INC(x) x=x+1 // Operaciones para modificar el iterador del bucle FOR
#define DEC(x) x=x-1

int yylex () ;
int yyerror () ;
char *mi_malloc (int) ;
char *gen_code (char *) ;
char *int_to_string (int) ;
char *char_to_string (char) ;
void añadir_variable_local(char *nombre) ;
int es_local(char *var_name) ;

char *local_variables[2048]; // Tabla de variables locales
int local_variables_counter = 0; // Contador de variable locales que tenemos
char dentro_funcion[500] = "global"; // Variable para saber en que scope nos encontramos
char temp [2048] ;
```

```
// Abstract Syntax Tree (AST) Node Structure
```

```
typedef struct ASTnode t_node ;
```

```
struct ASTnode {  
    char *op ;  
    int type ;      // leaf, unary or binary nodes  
    t_node *left ;  
    t_node *right ;  
} ;
```

```
// Definitions for explicit attributes
```

```
typedef struct s_attr {  
    int value ;      // - Numeric value of a NUMBER  
    char *code ;     // - to pass IDENTIFIER names, and other translations  
    t_node *node ;   // - for possible future use of AST  
} t_attr ;
```

```
#define YYSTYPE t_attr
```

```
%}
```

```
// Definitions for explicit attributes
```

```
%token NUMBER
```

```
%token IDENTIF      // Identificador=variable
```

```
%token INTEGER      // identifica el tipo entero
```

```

%token CHAR          // identifica el tipo char
%token FLOAT         //identifica el tipo float
%token STRING
%token MAIN          // identifica el comienzo del proc. main
%token WHILE         // identifica el bucle main
%token PUTS          // identifica la palabra clave puts para el print
%token PRINTF        // identifica la palabra clave printf para princ
%token AND           // identifica el operador &&
%token OR            // identifica el operador ||
%token NOT_EQUAL     // identifica el operador !=
%token EQUAL         // identifica el operador ==
%token LOE          //identifica el operador <=
%token GOE          //identifica el operador >=
%token IF           //identifica el operador IF
%token ELSE         //identifica el operador ELSE
%token FOR          // idenfifica el bucle FOR
%token INC          //identifica el operador INC
%token DEC          //identifica el operador DEC
%token SWITCH       //identifica la estructura de control switch
%token CASE         //identifica la palabra clave CASE del switch
%token BREAK        // identifica la palabra clave BREAK
%token DEFAULT      // identifica la palabra clave DEFAULT del switch
%token RETURN       // identifica la palabra clave RETURN para devolver valores en una función

// DEFINIMOS AQUI LA ASOCIATIVIDAD DE LOS OPERADORES

%right '='          // es la ultima operacion que se debe realizar
%left OR
%left AND
%left EQUAL NOT_EQUAL

```

```

%left '<' LOE '>' GOE
%left '+' '-'          // menor orden de precedencia
%left '*' '/' '%'      // orden de precedencia intermedio
%left UNARY_SIGN '!'    // mayor orden de precedencia

%%

// Seccion 3 Gramatica - Semantico

axioma:
    INTEGER dec_var ';'      {printf ("%s\n", $2.code); } //declaración de variables globales
    r_axioma                 {;}

    | dec_main '}'          {;}

    | dec_func '}' r_axioma {;}
    ;

// De momento sin parametros

dec_func:
    IDENTIF '(' func_params_declaration ')' '{' {strcpy(dentro_funcion, $1.code); local_variables_counter = 0;}
    r_sentencia {printf("(defun %s(%s)\n%s)\n", $1.code, $3.code, $7.code);
    strcpy(dentro_funcion, "global");}
    ;

func_params_declaration:
    // lambda, no tenemos params

```



```

func_params_call:
    // Puede ser que la función no tenga parámetros
    { $$code = gen_code(""); }

    | expresion func_params_call_cont    { sprintf(temp, "%s%s", $1.code, $2.code);
                                          $$code = gen_code(temp); }

    ;

func_params_call_cont:

    { $$code = gen_code(""); }

    | ',' expresion func_params_call_cont    { sprintf(temp, " %s%s", $2.code, $3.code);
                                              $$code = gen_code(temp); }

    ;

r_axioma:

    { ; }

    | axioma    { ; }

    ;

dec_main:
    MAIN '(' ')' '{'    { strcpy(dentro_funcion, "main"); local_variables_counter = 0; }
    r_sentencia          { printf("(defun %s ()\n%s)\n", $1.code, $6.code);
                          // Informamos que ya estamos dentro del main
    }

```

```

        strcpy(dentro_funcion,"global");}

;

r_sentencia:
        {$$.code = gen_code("");}

| sentencia r_sentencia      {sprintf(temp, "%s\n%s", $1.code, $2.code);
                              $$code = gen_code(temp);}

| RETURN expresion ';' r_sentencia {if(strlen($4.code) == 0){
                                     sprintf(temp,"%s", $2.code);
                                   }else{
                                     sprintf(temp,"(return-from %s %s)\n%s", dentro_funcion, $2.code, $4.code);
                                   }
                                   $$code = gen_code(temp);}

;

if_cont: '{' r_sentencia_anidada '}' else_cont {if(strlen($4.code) != 0){
                                                sprintf(temp,"\n(progn\n%s)\n(progn\n%s)", $2.code, $4.code);
                                              }else{
                                                sprintf(temp,"\n(progn\n%s)\n", $2.code);
                                              }
                                                $$code = gen_code(temp);}

;

r_sentencia_anidada:
        {$$.code = gen_code("");}

```

```

        | sentencia r_sentencia_anidada          {sprintf(temp, "%s\n%s", $1.code, $2.code);
                                                    $$code = gen_code(temp);}

        | RETURN expresion ';' r_sentencia_anidada {sprintf(temp, "(return-from %s %s)\n%s", dentro_funcion, $2.code,
$4.code);
                                                    $$code = gen_code(temp);}

        ;

else_cont:

                                                    {$$code = gen_code("");}

        | ELSE '{' r_sentencia_anidada '}' {sprintf(temp, "%s", $3.code);
                                                    $$code = gen_code(temp);}

        ;

sentencia:
    IDENTIF '=' expresion ';'
variables normales
                                                    {if (es_local($1.code)) { // Regla para asignaciones de
                                                    // Es local se le añade main_
                                                    sprintf(temp, "(setf %s_%s %s)", dentro_funcion, $1.code,
$3.code);

                                                    }else {
                                                    // Es global se usa el nombre de la variable original
                                                    sprintf(temp, "(setf %s %s)", $1.code, $3.code);
                                                    }
                                                    $$code = gen_code(temp);}

        | IDENTIF '[' expresion ']' '=' expresion ';'
                                                    {if (es_local($1.code)) { // Regla para índices de arrays

```



```

, $1.code, $3.code, $6.code);

$6.code);

| PRINTF '(' printf_param ')' ';'

| PUTS '(' STRING ')' ';'

| WHILE expresion while_cont

| IF expresion if_cont

| INTEGER dec_var ';'

| FOR '(' for_var ';' expr_condicional ';' for_operator

| SWITCH '(' IDENTIF ')' switch_cont

// Es local se le añade main_
sprintf(temp, "(setf (aref %s_%s %s) %s)", dentro_funcion

} else {
// Es global se usa el nombre de la variable original
sprintf(temp, "(setf (aref %s %s) %s)", $1.code, $3.code,

}
$$code = gen_code(temp);

{sprintf (temp, "%s", $3.code) ;
$$code = gen_code (temp) ; }

{sprintf(temp, "(print \"%s\")", $3.code);
$$code = gen_code(temp);}

{sprintf(temp, "(loop %s %s do\n%s)", $1.code, $2.code, $3.code);
$$code = gen_code(temp);}

{sprintf(temp, "(%s %s %s)", $1.code, $2.code, $3.code);
$$code = gen_code(temp);}

{$$code = $2.code;} // DECLARACIÓN DE VARIABLES LOCALES

{sprintf(temp, "%s\n(loop while %s do\n%s)", $3.code, $5.code,
$7.code);
$$code = gen_code(temp);}

{if (es_local($3.code)) {
// Es local se le añade main_

```

```

        sprintf(temp, "(case %s_%s\n%s)", dentro_funcion,
        $3.code, $5.code);
    }else{
        // Es global se usa el nombre de la variable original
        sprintf(temp, "(case %s\n%s)", $3.code, $5.code);
    }
    $$$.code = gen_code(temp);}

| func_call ';'
;

switch_cont:
    '{' CASE switch_val ':' r_sentencia_anidada BREAK ';' switch_cont2 '}' {sprintf(temp, "(%s
    \n%s)%s", $3.code, $5.code, $8.code);
    $$$.code = gen_code(temp);}
;

switch_cont2:
    {$$$.code = gen_code("");}

| DEFAULT ':' r_sentencia_anidada BREAK ';' {sprintf(temp, "(otherwise \n%s)\n", $3.code);
    $$$.code = gen_code(temp);}

| CASE switch_val ':' r_sentencia_anidada BREAK ';' switch_cont2 {sprintf(temp, "\n(%s \n%s)\n%s", $2.code,
    $4.code, $7.code);
    $$$.code = gen_code(temp);}
;

```

```

switch_val:
    '+' NUMBER %prec UNARY_SIGN    {sprintf(temp,"%d",$2.value);
                                   $$code = gen_code(temp); }

    | '-' NUMBER %prec UNARY_SIGN  {sprintf (temp, "(- %d)", $2.value);
                                   $$code = gen_code (temp) ; }

    | NUMBER                      {sprintf (temp, "%d", $1.value) ;
                                   $$code = gen_code (temp) ; }

    ;

for_operator:
    INC '(' IDENTIF ')' for_cont    {if(es_local($3.code)){
                                   sprintf(temp,"%s(setf %s_%s (+ %s_%s 1))", $6.code, dentro_funcion, $3.code,
                                   dentro_funcion, $3.code);
                                   }else{
                                   sprintf(temp,"%s(setf %s (+ %s 1))", $6.code, $3.code, $3.code);
                                   }
                                   $$code = gen_code(temp);
                                   }

    | DEC '(' IDENTIF ')' for_cont  {if(es_local($3.code)){
                                   sprintf(temp,"%s(setf %s_%s (- %s_%s 1))", $6.code, dentro_funcion, $3.code,
                                   dentro_funcion, $3.code);
                                   }else{
                                   sprintf(temp,"%s(setf %s (- %s 1))", $6.code, $3.code, $3.code);
                                   }
                                   $$code = gen_code(temp);
                                   }

```



```

while_cont:
    '{' r_sentencia_anidada '}' {$$code = $2.code;}
    ;

printf_param:
    STRING ',' printf_elem printf_cont {sprintf(temp,"%s%s", $3.code, $4.code);
                                         $$code = gen_code(temp);}
    ;

printf_elem:
    expresion {sprintf(temp, "(princ %s)", $1.code);
               $$code = gen_code(temp);}

    | STRING {sprintf(temp, "(princ \"%s\")", $1.code);
              $$code = gen_code(temp);}
    ;

printf_cont:
    {$$code = gen_code("");}

    | ',' printf_elem printf_cont {sprintf(temp, "\n%s%s", $2.code, $3.code);
                                    $$code = gen_code(temp);}
    ;

dec_var:
    IDENTIF continue_ID {if(strcmp(dentro_funcion,"global") == 0){

```

```

        sprintf(temp, "(setq %s %s", $1.code, $2.code);
    }else{
        sprintf(temp, "(setq %s_%s %s",dentro_funcion,$1.code, $2.code);
        añadir_variable_local($1.code);
    }
    $$code = gen_code(temp);}

```

```

;

```

continue_ID:

```

    continue_comma          {sprintf(temp, "%s", $1.code);
                             $$code = gen_code(temp);}

    | '=' NUMBER continue_comma {sprintf(temp, "%d%s", $2.value, $3.code);
                                $$code = gen_code(temp); }

    | '=' '-' NUMBER continue_comma {sprintf(temp, "-%d%s", $3.value, $4.code);
                                    $$code = gen_code(temp);}

    | '=' '+' NUMBER continue_comma {sprintf(temp, "%d%s", $3.value, $4.code);
                                    $$code = gen_code(temp);}

    | '[' expresion ']' continue_comma {sprintf(temp, "(make-array %s))%s", $2.code, $4.code);
                                       $$code = gen_code(temp);}

;

```

continue_comma:

```

    ',' dec_var      {sprintf(temp, "\n%s", $2.code);
                     $$code = gen_code(temp);}

```

```
|                {$$.code = "";}  
;
```

expresion:

```
    expr_condicional    {$$ = $1;}  
  
    | expr_others        {$$ = $1;}  
  
    | termino            {$$ = $1;}  
  
    | func_call           {$$ = $1;}  
  
    | array_index         {$$ = $1;}  
;
```

array_index: // Se utiliza para usar los contenidos del array

```
    IDENTIF '[' expresion ']'    {if (es_local($1.code)) { // Regla para índices de arrays  
                                // Es local se le añade main_  
                                sprintf(temp, "(aref %s_%s %s)", dentro_funcion , $1.code, $3.code);  
                                }else{  
                                // Es global se usa el nombre de la variable original  
                                sprintf(temp, "(aref %s %s)", $1.code, $3.code);  
                                }  
                                $$code = gen_code(temp);}  
  
    ;
```

expr_condicional:

expresion AND expresion	{sprintf (temp, "(and %s %s)", \$1.code, \$3.code) ; \$\$code = gen_code (temp);}
expresion OR expresion	{sprintf (temp, "(or %s %s)", \$1.code, \$3.code) ; \$\$code = gen_code (temp);}
expresion NOT_EQUAL expresion	{sprintf (temp, "(/= %s %s)", \$1.code, \$3.code) ; \$\$code = gen_code (temp);}
expresion EQUAL expresion	{sprintf (temp, "(= %s %s)", \$1.code, \$3.code) ; \$\$code = gen_code (temp);}
expresion '<' expresion	{sprintf (temp, "< %s %s)", \$1.code, \$3.code) ; \$\$code = gen_code (temp);}
expresion LOE expresion	{sprintf (temp, "(%s %s %s)", \$2.code, \$1.code, \$3.code) ; \$\$code = gen_code (temp);}
expresion '>' expresion	{sprintf (temp, "> %s %s)", \$1.code, \$3.code) ; \$\$code = gen_code (temp);}
expresion GOE expresion	{sprintf (temp, "(%s %s %s)", \$2.code, \$1.code, \$3.code) ; \$\$code = gen_code (temp);}
;	

expr_others:

expresion '%' expresion	{sprintf (temp, "(mod %s %s)", \$1.code, \$3.code) ; \$\$code = gen_code (temp) ;}
-------------------------	---


```

|   expresion '+' expresion      {sprintf (temp, "(+ %s %s)", $1.code, $3.code) ;
                                  $$code = gen_code (temp) ;}

|   expresion '-' expresion      {sprintf (temp, "(- %s %s)", $1.code, $3.code) ;
                                  $$code = gen_code (temp) ;}

|   expresion '*' expresion      {sprintf (temp, "(* %s %s)", $1.code, $3.code) ;
                                  $$code = gen_code (temp) ;}

|   expresion '/' expresion      {sprintf (temp, "(/ %s %s)", $1.code, $3.code) ;
                                  $$code = gen_code (temp) ;}

;

```

termino:

```

operando                          {$$ = $1;}

|   '+' operando %prec UNARY_SIGN  {$$ = $1;}

|   '-' operando %prec UNARY_SIGN  {sprintf (temp, "(- %s)", $2.code);
                                  $$code = gen_code (temp);}

|   '!' operando                  {sprintf (temp, "(not %s)", $2.code) ;
                                  $$code = gen_code (temp);}

;

```

operando:

```

IDENTIF                          {if (es_local($1.code)) {
                                  sprintf(temp, "%s_%s", dentro_funcion,$1.code);

```

```

        }else{
            sprintf(temp, "%s", $1.code);
        }
        $$code = gen_code(temp);}

|    NUMBER            {sprintf (temp, "%d", $1.value) ;
                        $$code = gen_code (temp) ; }

|    '(' expresion ')'  { $$ = $2 ; }
;

```

```

%%                                     // SECCION 4    Codigo en C

```

```

int n_line = 1 ;

```

```

int yyerror (mensaje)

```

```

char *mensaje ;

```

```

{
    fprintf (stderr, "%s en la linea %d\n", mensaje, n_line) ;
    printf ( "\n") ;    // bye
}

```

```

char *int_to_string (int n)

```

```

{
    char ltemp [2048] ;

    sprintf (ltemp, "%d", n) ;

    return gen_code (ltemp) ;
}

```

```

}

char *char_to_string (char c)
{
    char ltemp [2048] ;

    sprintf (ltemp, "%c", c) ;

    return gen_code (ltemp) ;
}

char *my_malloc (int nbytes)          // reserva n bytes de memoria dinamica
{
    char *p ;
    static long int nb = 0;           // sirven para contabilizar la memoria
    static int nv = 0 ;               // solicitada en total

    p = malloc (nbytes) ;
    if (p == NULL) {
        fprintf (stderr, "No queda memoria para %d bytes mas\n", nbytes) ;
        fprintf (stderr, "Reservados %ld bytes en %d llamadas\n", nb, nv) ;
        exit (0) ;
    }
    nb += (long) nbytes ;
    nv++ ;

    return p ;
}

```

```
/*  
***** Seccion de Palabras Reservadas *****  
*/
```

```
typedef struct s_keyword { // para las palabras reservadas de C  
    char *name ;  
    int token ;  
} t_keyword ;
```

```
t_keyword keywords [] = { // define las palabras reservadas y los  
    "main",          MAIN,          // y los token asociados  
    "while",         WHILE,  
    "int",           INTEGER,  
    "char",          CHAR,  
    "float",         FLOAT,  
    "puts",          PUTS,  
    "printf",        PRINTF,  
    "&&",            AND,  
    "||",           OR,  
    "!=",          NOT_EQUAL,  
    "==",          EQUAL,  
    "<=",          LOE,  
    ">=",          GOE,  
    "if",          IF,  
    "else",        ELSE,  
    "for",         FOR,  
    "inc",         INC,  
    "dec",         DEC,  
    "switch",      SWITCH,  
    "case",        CASE,
```

```

    "break",      BREAK,
    "default",    DEFAULT,
    "return",     RETURN,
    NULL,         0 // para marcar el fin de la tabla
} ;

int es_local(char *var_name){
    for(int i = 0; i < local_variables_counter; i++){
        if(strcmp(var_name,local_variables[i]) == 0){
            // Si coincide el nombre de la variable con alguna del array, entonces es local.
            return 1;
        }
    }
    // Si no hubo ninguna coincidencia, entonces es global
    return 0;
}

void añadir_variable_local(char *nombre){
    // Añadimos la variable al array y aumentamos el contador
    local_variables[local_variables_counter] = gen_code(nombre);
    local_variables_counter++;
}

t_keyword *search_keyword (char *symbol_name)
{
    // Busca n_s en la tabla de pal. res.
    // y devuelve puntero a registro (simbolo)

    int i ;
    t_keyword *sim ;

```

```

i = 0 ;
sim = keywords ;
while (sim [i].name != NULL) {
    if (strcmp (sim [i].name, symbol_name) == 0) {
        // strcmp(a, b) devuelve == 0 si a==b
        return &(sim [i]) ;
    }
    i++ ;
}

return NULL ;
}

/*****/
/**** Seccion del Analizador Lexicografico *****/
/*****/

char *gen_code (char *name)    // copia el argumento a un
{                               // string en memoria dinamica
    char *p ;
    int l ;

    l = strlen (name)+1 ;
    p = (char *) my_malloc (l) ;
    strcpy (p, name) ;

    return p ;
}

```

```

int yylex ()
{
    // NO MODIFICAR ESTA FUNCION SIN PERMISO
    int i ;
    unsigned char c ;
    unsigned char cc ;
    char ops_expandibles [] = "!<=|>%&/+-*" ;
    char temp_str [256] ;
    t_keyword *symbol ;

    do {
        c = getchar () ;

        if (c == '#') { // Ignora las lineas que empiezan por # (#define, #include)
            do { // OJO que puede funcionar mal si una linea contiene #
                c = getchar () ;
            } while (c != '\n') ;
        }

        if (c == '/') { // Si la linea contiene un / puede ser inicio de comentario
            cc = getchar () ;
            if (cc != '/') { // Si el siguiente char es / es un comentario, pero...
                ungetc (cc, stdin) ;
            } else {
                c = getchar () ; // ...
                if (c == '@') { // Si es la secuencia //@ ==> transcribimos la linea
                    do { // Se trata de codigo inline (Codigo embebido en C)
                        c = getchar () ;
                        putchar (c) ;
                    } while (c != '\n') ;
                }
            }
        }
    } while (c != '\n') ;
}

```

```

        } while (c != '\n') ;
    } else {          // ==> comentario, ignorar la linea
        while (c != '\n') {
            c = getchar () ;
        }
    }
}

} else if (c == '\\') c = getchar () ;

if (c == '\n')
    n_line++ ;

} while (c == ' ' || c == '\n' || c == 10 || c == 13 || c == '\t') ;

if (c == '\"') {
    i = 0 ;
    do {
        c = getchar () ;
        temp_str [i++] = c ;
    } while (c != '\"' && i < 255) ;
    if (i == 256) {
        printf ("AVISO: string con mas de 255 caracteres en linea %d\n", n_line) ;
    }
    // habria que leer hasta el siguiente " , pero, y si falta?
    temp_str [--i] = '\0' ;
    yylval.code = gen_code (temp_str) ;
    return (STRING) ;
}

if (c == '.' || (c >= '0' && c <= '9')) {
    ungetc (c, stdin) ;

```



```

        scanf ("%d", &yylval.value) ;
//      printf ("\nDEV: NUMBER %d\n", yylval.value) ;      // PARA DEPURAR
        return NUMBER ;
    }

    if ((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z')) {
        i = 0 ;
        while (((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z') ||
            (c >= '0' && c <= '9') || c == '_' ) && i < 255) {
            temp_str [i++] = tolower (c) ;
            c = getchar () ;
        }
        temp_str [i] = '\0' ;
        ungetc (c, stdin) ;

        yylval.code = gen_code (temp_str) ;
        symbol = search_keyword (yylval.code) ;
        if (symbol == NULL) {      // no es palabra reservada -> identificador antes vrvariable
//      printf ("\nDEV: IDENTIF %s\n", yylval.code) ;      // PARA DEPURAR
            return (IDENTIF) ;
        } else {
//      printf ("\nDEV: OTRO %s\n", yylval.code) ;      // PARA DEPURAR
            return (symbol->token) ;
        }
    }

    if (strchr (ops_expandibles, c) != NULL) { // busca c en ops_expandibles
        cc = getchar () ;
        sprintf (temp_str, "%c%c", (char) c, (char) cc) ;
        symbol = search_keyword (temp_str) ;
    }

```

```

    if (symbol == NULL) {
        ungetc (cc, stdin) ;
        yylval.code = NULL ;
        return (c) ;
    } else {
        yylval.code = gen_code (temp_str) ; // aunque no se use
        return (symbol->token) ;
    }
}

// printf ("\nDEV: LITERAL %d %#c#\n", (int) c, c) ; // PARA DEPURAR
if (c == EOF || c == 255 || c == 26) {
// printf ("tEOF ") ; // PARA DEPURAR
    return (0) ;
}

return c ;
}

int main ()
{
    yyparse () ;
}

```